

Shell综合应用实战

警示：在Shell程序中，当使用到和路径有关的信息时，全部使用 **绝对路径**

1. 自动化web服务器日志分析

1.1 需求

目标: 编写一个 Shell 脚本，自动分析指定日期的 Web 服务器访问日志，提取基本统计信息。

场景: 假设我们的 Web 服务器（如 Apache 或 Nginx）每天生成一个访问日志文件，格式为 `access.log.YYYY-MM-DD`，存储在 `/var/log/webserver/` 目录下。我们需要一个脚本，输入日期，然后分析对应日期的日志文件。

日志格式：

```
1 | <IP> <ident> <user> [DD/Mon/YYYY:HH:MM:SS +ZZZZ] "METHOD  
  | /path HTTP/1.X" STATUS BYTES "Referer" "User-Agent"
```

- `$1` : IP 地址
- `$4` : 时间戳 (带方括号)
- `$5` : 时区
- `$6` : 请求方法 (GET, POST)
- `$7` : 请求路径
- `$8` : HTTP 协议
- `$9` : HTTP 状态码
- `$10` : 返回字节数

具体需求: 统计总请求数、不同状态码的请求数、不同请求方法的请求数、请求量最多的前5个IP地址

1. **日志文件检查:** 脚本会检查要分析的日志文件是否存在，如果不存在则输出错误信息并退出。

2. 总请求数统计：使用 `wc -l` 命令统计日志文件的行数，即总请求数。
3. 不同状态码的请求数统计：使用 `awk` 提取日志中的状态码字段，再通过 `sort`、`uniq -c` 和 `sort -nr` 进行排序和计数。
4. 不同请求方法的请求数统计：使用 `awk` 提取请求方法字段，经过 `cut` 命令处理后，再进行排序和计数。
5. 请求量最多的前 5 个 IP 地址：使用 `awk` 提取 IP 地址字段，排序、计数后取前 5 个。
6. 每天的请求数统计：使用 `awk` 提取日期字段，排序、计数后按日期排序。

1.2 数据样本

```
1 # 创建日志目录
2 log_dir="/var/log/webserver"
3 mkdir -p "$log_dir"
4
5 # 生成模拟日志文件 (access.log.2025-04-06)
6 cat << EOF > "${log_dir}/access.log.2025-04-06"
7 192.168.1.10 - - [06/Apr/2025:10:15:30 +0800] "GET
   /index.html HTTP/1.1" 200 1543 "-" "Mozilla/5.0 (Windows
   NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
   Gecko) Chrome/90.0.4430.93 Safari/537.36"
8 192.168.1.11 - - [06/Apr/2025:10:15:35 +0800] "GET
   /images/logo.png HTTP/1.1" 200 5678
   "http://example.com/index.html" "Mozilla/5.0 (iPhone; CPU
   iPhone OS 14_4 like Mac OS X) AppleWebKit/605.1.15 (KHTML,
   like Gecko) Version/14.0.3 Mobile/15E148 Safari/604.1"
9 192.168.1.10 - - [06/Apr/2025:10:16:01 +0800] "POST /login
   HTTP/1.1" 302 120 "http://example.com/login.html"
   "Mozilla/5.0 (Windows NT 10.0; Win64; x64) ..."
10 192.168.1.12 - - [06/Apr/2025:10:17:05 +0800] "GET
   /admin/dashboard HTTP/1.1" 403 210 "-" "curl/7.68.0"
11 10.0.0.5 - - [06/Apr/2025:10:18:10 +0800] "GET
   /api/data?id=123 HTTP/1.1" 500 50 "-"
   "InternalService/1.0"
12 192.168.1.10 - - [06/Apr/2025:10:18:15 +0800] "GET
   /index.html HTTP/1.1" 200 1543 "-" "Mozilla/5.0 (Windows
   NT 10.0; Win64; x64) ..."
```

```
13 192.168.1.13 - - [06/Apr/2025:10:19:00 +0800] "GET
    /styles.css HTTP/1.1" 200 890
    "http://example.com/index.html" "Mozilla/5.0 (Windows NT
    10.0; Win64; x64) ..."
14 192.168.1.11 - - [06/Apr/2025:10:20:22 +0800] "GET
    /images/logo.png HTTP/1.1" 304 -
    "http://example.com/index.html" "Mozilla/5.0 (iPhone; CPU
    iPhone OS 14_4 like Mac OS X) ..."
15 10.0.0.5 - - [06/Apr/2025:10:21:30 +0800] "GET
    /api/data?id=456 HTTP/1.1" 503 50 "-"
    "InternalService/1.0"
16 192.168.1.12 - - [06/Apr/2025:10:22:00 +0800] "GET
    /forbidden HTTP/1.1" 403 210 "-" "curl/7.68.0"
17 EOF
18
19 # 生成另一个日期的日志 (access.log.2025-04-05)
20 cat << EOF > "${log_dir}/access.log.2025-04-05"
21 172.16.0.5 - - [05/Apr/2025:08:05:10 +0800] "GET /home
    HTTP/1.1" 200 2048 "-" "Mozilla/5.0 ..."
22 172.16.0.6 - - [05/Apr/2025:08:06:20 +0800] "POST /submit
    HTTP/1.1" 200 50 "http://othersite.com/" "Firefox/88.0"
23 172.16.0.5 - - [05/Apr/2025:08:07:00 +0800] "GET /home
    HTTP/1.1" 304 - "-" "Mozilla/5.0 ..."
24 172.16.0.7 - - [05/Apr/2025:08:08:30 +0800] "GET
    /notfound.html HTTP/1.1" 404 300 "-" "Chrome/..."
25 EOF
```

1.3 代码实现

编写Shell程序: `web_log_analysis.sh`

```
1  #!/bin/bash
2
3  # 配置信息: 日志文件路径
4  LOG_DIR="/var/log/webserver"
5  LOG_PREFIX="access.log"
6
7  target_date="$1"      # 接收脚本程序传递的第1个参数
8
```

```

9 # 日志文件路径
10 LOG_FILE="${LOG_DIR}/${LOG_PREFIX}.${target_date}"
11
12
13 # 函数：记录错误日志
14 function log_error() {
15     # local 的作用是限定变量仅能在当前函数中使用
16     local message="$1"
17     # $(命令..) 用于在shell中执行linux中的命令
18     echo "[ERROR] $(date '+%Y-%m-%d %H:%M:%S') - $message"
19     >&2
20 }
21
22 # 函数：日期格式校验
23 function checkDateFormat(){
24     # 使用 =~ 来判断一个变量是否匹配指定的正则表达式
25     if ! [[ "$target_date" =~ ^[0-9]{4}-[0-9]{2}-[0-9]{2}$
26     ]]; then
27         # 调用log_error函数，并传递参数
28         log_error "日期格式无效，请使用YYYY-MM-DD日期格式。"
29         echo "使用：$0 <YYYY-MM-DD>" >&2
30         echo "      分析指定日期的web服务器日志文件。" >&2
31         exit 1 # exit 0：正常运行程序并退出程序 exit 1
32         : 非正常运行导致退出程序
33     fi
34 }
35
36 # 函数：检查日志文件是否存在
37 function checkLogfileExists(){
38     if [ ! -f "$LOG_FILE" ]; then
39         echo "错误：日志文件 $LOG_FILE 未找到。"
40         exit 1
41     fi
42 }
43
44 # 函数：需求1 - 统计总请求数
45 function totalRequest(){
46     total_requests=$(wc -l < "$LOG_FILE")
47     echo "总请求数：$total_requests"
48 }

```

```
47 # 函数：需求2 - 统计不同状态码的请求数
48 function httpStatusRequest(){
49     echo "不同状态码的请求数："
50     awk '{print $9}' "$LOG_FILE" | sort | uniq -c | sort -
nr
51 }
52
53 # 函数：需求3 - 统计不同请求方法的请求数
54 function httpMethodRequest(){
55     echo "不同请求方法的请求数："
56     awk '{print $6}' "$LOG_FILE" | cut -d'"'"' -f2 | cut -d'
' -f1 | sort | uniq -c | sort -nr
57 }
58
59
60 # 函数：需求4 - 找出请求量最多的前5个IP地址
61 function httpRequestCountTop5(){
62     echo "请求量最多的前 5 个 IP 地址："
63     awk '{print $1}' "$LOG_FILE" | sort | uniq -c | sort -
nr | head -n 5
64 }
65
66 # 函数：需求5 - 统计每天的请求数
67 function httpDayRequest(){
68     echo "每天的请求数："
69     awk '{print substr($4, 2, 11)}' "$LOG_FILE" | sort |
uniq -c | sort -k2
70 }
71
72 # 主函数：调用其他函数执行
73 function app(){
74     checkDateFormat      # 检查日期格式是否合规
75
76     checkLogfileExists   # 检查日志文件是否存在
77     totalRequest          # 统计总请求数
78     httpStatusRequest     # 统计不同状态码的请求数
79     httpMethodRequest     # 统计不同请求方法的请求数
80     httpRequestCountTop5 # 找出请求量最多的前5个IP地址
81     httpDayRequest        # 统计每天的请求数
82 }
83
```

```
84 # 调用主函数
85 app
```

2. 系统监控

2.1 需求

目标: 创建一个 Shell 脚本 `system_monitor.sh`，用于监控 CPU、内存和指定文件系统的磁盘使用率，当超过预设阈值时发出告警。

脚本特性:

- 可配置: 通过配置文件 `config.conf` 设置监控阈值和检查的文件系统。
- 模块化: 使用函数分别检查 CPU、内存和磁盘。
- 告警: 当资源使用率超过阈值时，打印告警信息到标准错误，并写入告警日志文件。
- 日志记录: 记录每次检查的结果和发出的告警。

2.2 需求拆解

需求1: 通过配置文件 `config.conf` 设置监控阈值和检查的文件系统。

- 配置文件: `config.conf`

```
1 # 配置信息
2 # CPU的阈值
3 变量1=80
4 # 内存的阈值
5 变量2=80
6 # 硬盘的阈值
7 变量3=80
8
9 # 文件磁盘
```

```
10 FILEDIDK="/"
```

- Shell脚本程序文件: `system_monitor.sh`

```
1 # 配置文件路径
2 CONFIG_FILE="/export/config.conf" ## 注意: 使用绝对路径
3
4 # 判断配置文件是否存在
5 if [ ! -f "$CONFIG_FILE" ]; then
6     echo "[$(date '+%Y-%m-%d %H:%M:%S')] 错误: 配置文件
7     $CONFIG_FILE 未找到。" >&2
8     exit 1 # 如果文件不存在则退出程序
9 fi
10 # 配置文件存在, 则加载该配置文件
11 source "$CONFIG_FILE"
```

需求2: 检查 CPU

```
1 # 获取 CPU 使用率
2 CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print $2 +
3 $4}')
4 # 检查 CPU 使用率
5 if (( $(awk "BEGIN{print ($CPU_USAGE > $CPU_THRESHOLD)? 1 :
6 0}") )); then
7     echo "警告: CPU 使用率超过 $CPU_THRESHOLD%, 当前使用率为
8     $CPU_USAGE%"
9 fi
```

- `awk "BEGIN{print ($cpu_usage > $CPU_THRESHOLD)? 1 : 0}" : awk 的 BEGIN 块在处理输入文件之前执行，这里使用三元运算符 ? : 来判断 $cpu_usage 是否大于 $CPU_THRESHOLD，若大于则输出 1，否则输出 0。`
- 三元运算符：

```

1 | 在shell程序中可以使用：双支if语句
2 | 变量x
3 | if [[ 10 > 5 ]] then
4 |     变量x=结果1
5 | else
6 |     变量x=结果2
7 | fi
8 | # 以上代码可以使用三元运算符简化写法          变量 = 条件 ?
   | 结果1 : 结果2
9 | 变量x= 10>5 ? 结果1 : 结果2

```

需求3：检查内存

```

1 | # 获取内存使用率
2 | MEMORY_USAGE=$(free | awk '/Mem/{printf("%.0f")', $3*100/$2
   | })
3 |
4 | # 检查内存使用率
5 | if (( $MEMORY_USAGE > $MEMORY_THRESHOLD )); then
6 |     echo "警告：内存使用率超过 $MEMORY_THRESHOLD%，当前使用率
   | 为 $MEMORY_USAGE%"
7 | fi

```

- `free` 是一个 Linux 系统自带的命令，用于显示系统内存的使用情况，包括物理内存、交换空间等。执行该命令后，会输出类似以下的信息：

	total	used	free	shared
1 buff/cache	available			
2 Mem:	8172444	3004200	2536732	53376
	2631512	4733020		
3 Swap:	2097148	0	2097148	

- 其中，`Mem` 行显示了物理内存的使用情况，包含总内存（`total`）、已使用内存（`used`）、空闲内存（`free`）等信息。

需求4：检查磁盘

```
1 # 获取磁盘使用率
2 DISK_USAGE=$(df -h $FILESYSTEM | awk 'NR==2 {print
  substr($5, 1, length($5)-1)}')
3
4 # 检查磁盘使用率
5 if (( $DISK_USAGE > $DISK_THRESHOLD )); then
6     echo "警告: $FILESYSTEM 文件系统磁盘使用率超过
  $DISK_THRESHOLD%, 当前使用率为 $DISK_USAGE%"
7 fi
```

- `df -h` 是linux中常用的命令，用于查看文件系统的磁盘使用情况。
- 在终端中执行 `df -h` 命令，输出结果可能如下：

```
1 | Filesystem      Size  Used Avail Use% Mounted on
2 | /dev/sda1        20G   10G   8.0G  56% /
3 | devtmpfs         7.8G    0   7.8G   0% /dev
4 | tmpfs            7.9G    0   7.9G   0% /dev/shm
5 | tmpfs            7.9G  9.8M   7.9G   1% /run
6 | tmpfs            7.9G    0   7.9G   0% /sys/fs/cgroup
7 | /dev/sdb1        100G   20G   80G  20% /data
```

字段解释：

- **Filesystem**：代表文件系统的名称，通常是磁盘分区或者存储设备的标识。
- **Size**：表示文件系统的总容量。
- **Used**：指已使用的磁盘空间大小。
- **Avail**：表示可用的磁盘空间大小。
- **Use%**：是已使用磁盘空间所占的百分比。
- **Mounted on**：表示文件系统的挂载点，也就是文件系统在系统目录树中的挂载位置。

2.2 代码实现

1. 创建配置文件： `config.conf`

```
1 # 预设阈值
```

```
2 CPU_THRESHOLD=80      # CPU
3 MEMORY_THRESHOLD=80   # 内存
4 DISK_THRESHOLD=80     # 磁盘
5
6 # 指定要监控的文件系统
7 FILESYSTEM="/"
```

2. 编写监控脚本: `system_monitor.sh`

```
1 #!/bin/bash
2
3 # 配置文件路径
4 CONFIG_FILE="/export/config.conf"  ## 注意: 使用绝对路径
5 # 日志文件路径
6 LOG_FILE="/export/resource_monitor.log" ##注意: 使用绝对路径
7
8 # 加载配置文件
9 load_config() {
10     if [ ! -f "$CONFIG_FILE" ]; then
11         echo "[$(date '+%Y-%m-%d %H:%M:%S')] 错误: 配置文件
$CONFIG_FILE 未找到。" >&2
12         exit 1
13     fi
14     source "$CONFIG_FILE"
15 }
16
17 # 检查 CPU 使用率
18 check_cpu() {
19     local cpu_usage=$(top -bn1 | grep "Cpu(s)" | awk
'{print $2 + $4}')
20     echo "[$(date '+%Y-%m-%d %H:%M:%S')] CPU 检查结果: 当前
使用率为 $cpu_usage%" >> "$LOG_FILE"
21
22     if (( $(awk "BEGIN{print ($cpu_usage >
$CPU_THRESHOLD)? 1 : 0}") )); then
23         local alert_msg="警告: CPU 使用率超过
$CPU_THRESHOLD%, 当前使用率为 $cpu_usage%"
24         echo "$alert_msg" >&2
25         echo "[$(date '+%Y-%m-%d %H:%M:%S')] - $alert_msg"
>> "$LOG_FILE"
```

```
26     fi
27 }
28
29 # 检查内存使用率
30 check_memory() {
31     local memory_usage=$(free | awk '/Mem/{printf("%.0f"),
32 $3*100/$2 }')
33     echo "[$(date '+%Y-%m-%d %H:%M:%S')] 内存检查结果: 当前
34 使用率为 $memory_usage%" >> "$LOG_FILE"
35
36     if (( $memory_usage > $MEMORY_THRESHOLD )); then
37         local alert_msg="警告: 内存使用率超过
38 $MEMORY_THRESHOLD%, 当前使用率为 $memory_usage%"
39         echo "$alert_msg" >&2
40         echo "$(date '+%Y-%m-%d %H:%M:%S') - $alert_msg"
41 >> "$LOG_FILE"
42     fi
43 }
44
45 # 检查磁盘使用率
46 check_disk() {
47     local disk_usage=$(df -h "$FILESYSTEM" | awk 'NR==2
48 {print substr($5, 1, length($5)-1)}')
49     echo "[$(date '+%Y-%m-%d %H:%M:%S')] 磁盘检查结果:
50 $FILESYSTEM 文件系统当前使用率为 $disk_usage%" >>
51 "$LOG_FILE"
52
53     if (( $disk_usage > $DISK_THRESHOLD )); then
54         local alert_msg="警告: $FILESYSTEM 文件系统磁盘使用
55 率超过 $DISK_THRESHOLD%, 当前使用率为 $disk_usage%"
56         echo "$alert_msg" >&2
57         echo "$(date '+%Y-%m-%d %H:%M:%S') - $alert_msg"
58 >> "$LOG_FILE"
59     fi
60 }
61
62 # 主函数
63 main() {
64     load_config
65     check_cpu
66     check_memory
67 }
```

```
58     check_disk
59 }
60
61 main
```

3. 任务调度

3.1 概述

任务调度通俗来讲，就是对各种任务的执行时间、顺序等进行合理安排和管理，就像学校的课程表一样，给不同的课程(任务)安排合适的时间和顺序。

生活中的例子

- 1 在日常生活里，任务调度很常见。比如家庭主妇安排一天的家务任务，早上先把衣服放进洗衣机洗，在洗衣服的同时打扫房间，等衣服洗好再晾晒，接着准备午饭。这里洗衣服、打扫房间、准备午饭就是不同的任务，主妇根据任务的特点和所需时间，合理安排它们的执行顺序和时间，这就是一种简单的任务调度。

计算机领域

- 1 在计算机领域，任务调度也非常重要。计算机要同时处理很多任务，像运行程序、处理数据、响应用户操作等。操作系统里的任务调度器就负责对这些任务进行管理。

- **时间安排：**它会给每个任务分配一定的 CPU 时间片。例如，当你同时打开浏览器、音乐播放器和文档编辑器时，操作系统会轮流让这些程序使用 CPU 资源，让每个程序都有机会运行。比如浏览器先运行一段时间，然后音乐播放器运行，接着文档编辑器运行，如此循环，让你感觉这些程序好像在同时运行。
- **优先级设定：**还会根据任务的重要性和紧急程度设定优先级。重要且紧急的任务会优先处理。比如，当计算机正在进行复杂的图形渲染任务

时，突然收到一个系统更新的请求，如果系统更新的优先级设置得高，任务调度器就会暂停图形渲染任务，先处理系统更新。

应用场景举例

- 1 任务调度在很多领域都有应用，比如在服务器上，会有很多定时任务，像每天凌晨备份数据、每周统计网站访问量等。
- 2 服务器的任务调度系统会按照预设的时间准确执行这些任务，保证数据的安全性和业务的正常运行。

3.2 linux中的任务调度

在 Linux 系统里，任务调度是一项重要功能，它能让用户按照预先设定的时间、间隔或条件来自动执行特定任务。这极大地提升了系统管理的效率，也让许多重复性工作得以自动化。下面我们来学习一个 Linux 中常用的任务调度工具：`cron`。

`cron` 是 Linux 系统中最常用的任务调度工具，它能依据用户设定的时间规则周期性地执行任务。

核心概念：

- `cron`：在后台运行的守护进程。
- `crontab` (Cron Table): 每个用户都可以拥有自己的 `crontab` 文件，用于定义该用户的定时任务列表。系统也有一个全局的 `/etc/crontab` 文件和 `/etc/cron.d/` 目录。

`crontab` 文件格式：

- `crontab` 文件中的每一行代表一个定时任务，格式如下：

```
1 | .----- minute (0 - 59)
2 | | .----- hour (0 - 23)
3 | | | .----- day of month (1 - 31)
4 | | | | .----- month (1 - 12) OR jan,feb,mar,apr...
5 | | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR
   | | | | | sun,mon,tue,wed,thu,fri,sat
6 | | | | |
```

特殊字符:

- `*`: 代表该时间单位的所有可能值 (例如, 分钟位是 `*` 表示每分钟)。
- `,`: 用于分隔多个值 (例如, `1,15,30` 表示第 1、15、30 分钟)。
- `-`: 用于表示一个范围 (例如, `9-17` 表示 9 点到 17 点)。
- `/`: 用于表示步长 (例如, `*/15` 在分钟位表示每 15 分钟, 等同于 `0,15,30,45`)。

表格: `crontab` 格式详解与示例

字段	允许值	特殊字符含义示例
分钟 (Minute)	0-59	<code>*</code> (每分钟); <code>0</code> (每小时的 0 分); <code>*/10</code> (每 10 分钟)
小时 (Hour)	0-23	<code>*</code> (每小时); <code>8</code> (早上 8 点); <code>9-17</code> (9 点到 17 点)
日 (DayOfMonth)	1-31	<code>*</code> (每天); <code>1</code> (每月 1 号); <code>1,15</code> (每月 1 号和 15 号)
月 (Month)	1-12(or names)	<code>*</code> (每月); <code>1</code> (一月); <code>jun,jul,aug</code> (六、七、八月)
星期 (DayOfWeek)	0-7	<code>*</code> (每天); <code>1</code> (周一); <code>1-5</code> (周一到周五); <code>0,6</code> (周日和周六)

常用 `crontab` 命令:

- `crontab -e` : 编辑 当前用户的 `crontab` 文件。第一次执行时会提示选择编辑器 (如 `vim` , `nano`)。
- `crontab -l` : 列出 当前用户的 `crontab` 文件内容。
- `crontab -r` : 移除 当前用户的 `crontab` 文件 (危险操作! 会删除所有定时任务!)。

3.3 示例分析

假设我们希望脚本在每天凌晨 1:05 执行，分析前一天的日志。

1. 编辑 crontab:

```
1 | crontab -e
```

2. 添加任务行:

在打开的编辑器中，添加类似下面的一行（请务必使用脚本的绝对路径！）：

```
1 | # 每天凌晨1:05运行日志分析器，获取前一天的日志
2 | 5 1 * * * /home/bigdata/scripts/log_analyzer.sh $(date -d
   | "yesterday" +\%Y-\%m-\%d) >>
   | /home/bigdata/logs/log_analyzer_cron.log 2>&1
```

命令解释:

- `5 1 * * *` : 指定执行时间为每天的 1 点 05 分。
- `/home/bigdata/scripts/log_analyzer.sh` : 脚本的绝对路径 (假设脚本放在这里)。
- `$(date -d "yesterday" +\%Y-\%m-\%d)` :
 - `date -d "yesterday"` : 获取昨天的日期
 - `+\%Y-\%m-\%d` : 将日期格式化为 `YYYY-MM-DD`
 - 注意：在 `crontab` 中，`%` 符号有特殊含义（表示换行），必须用反斜杠 `\` 进行转义！
 - `$(...)` : 命令替换，将计算出的日期作为参数传递给脚本

- `>> /home/bigdata/logs/log_analyzer_cron.log` : 将脚本的标准输出追加到日志文件。
- `2>&1` : 将标准错误也重定向到与标准输出相同的地方（即追加到日志文件）。这非常重要，可以捕获脚本执行过程中的所有输出和错误信息。

强烈建议: 在 `crontab` 中执行的脚本，全部使用绝对路径，并确保所有依赖的环境变量都在脚本内部设置或通过 `source` 加载。同时，务必重定向输出和错误到日志文件以便排查问题。

3.4 实现"系统监控"的任务调度

通过linux的任务调度工具 `crontab`，实现每隔1分执行一次系统监控的Shell脚本程序：

```
1 | # 第1步：编辑 crontab
2 | crontab -e
3 |
4 | # 第2步：添加新任务
5 | */1 * * * * /export/system_monitor.sh
```

设置完成后可以通过：`tail -f resource_monitor.log`，动态查看日志文件内容，来确定任务调度执行是否正常。

附录1

在shell脚本中，默认情况下，总是有三个文件处于打开状态：

- 标准输入(键盘输入)，对应描述符数字：`0`
- 标准输出（默认输出到屏幕）对应描述符数字：`1`
- 标准错误（默认输出到屏幕）对应描述符数字：`2`

`>` 默认为标准输出重定向，与 `1>` 相同


```
1 | echo "Hello" > file.log
2 | echo "Hello" 1> file.log
```

& 是一个描述符，如果1或2前不加&，会被当成一个普通文件。

&>filename 意思是把标准输出和标准错误输出都重定向到文件filename中。

2>&1 意思是把标准输出和标准错误输出都定向到标准输出

```
1 | echo "hell" 2>&1
```

>&2 意思是把标准输出重定向到标准错误

```
1 | echo "info..." >&2
```

附录2: Linux扩展命令

sort命令

sort 可针对文本文件的内容，以行为单位来排序。（默认排序规则：按字符升序）

命令格式: **sort** [选项] 文件名

示例1: 对字符串排序

```
1 | ## 准备工作: 创建1.txt文件
2 | banana
3 | apple
4 | pear
5 | orange
6 | pear
```

```
1 | sort 1.txt
```

示例2：去重排序

选项	英文	含义
-u	unique	去掉重复的

```
1 | sort -u 1.txt
```

示例3：对数值排序

选项	英文	含义
-n	numeric-sort	按照数值大小排序
-r	reverse	使次序颠倒

```
1 | ## 准备工作：创建2.txt文件
2 | 1
3 | 3
4 | 5
5 | 7
6 | 11
7 | 2
8 | 4
9 | 6
10 | 10
11 | 8
12 | 9
```

```
1 | # 可以先使用：sort 2.txt 看看是什么结果
2 | sort -n 2.txt # 默认升序
3 | sort -n -r 2.txt # 把升序变为降序
```

示例4：对成绩排序

选项	英文	含义
-t	field-separator	指定字段分隔符

选项	英文	含义
-k	key	根据那一列排序

```

1 | 准备工作：创建 score.txt文件
2 | zs 68 99 26
3 | ls 98 66 96
4 | ww 38 33 86
5 | zl 78 44 36
6 | we 88 22 66
7 | zb 98 44 46

```

```

1 | # 根据第二段成绩 进行倒序显示所有内容
2 | sort -t ' ' -k2nr score.txt

```

cut命令

cut 根据条件从命令结果中提取对应内容

命令格式：

```

1 | cut 选项 文件
2 | 或
3 | 命令结果 | cut [选项]

```

示例1：截取出文件中前2行的第5个字符

选项	英文	含义
-c	characters	按字符选取内容

```

1 | ## 准备工作： 创建3.txt文件
2 | 111:aaa:bbb:ccc
3 | 222:ddd:eee:fff
4 | 333:ggg:hhh

```

```
1 # 分析：截取出3.txt文件中前2行
2 head -2 3.txt
3
4 # 分析：截取出3.txt文件中前2行第5个字符
5 head -2 3.txt | cut -c 5
6
```

示例2：截取出文件中前2行以“:”进行分割的第1,2段内容

选项	英文	含义
-d '分隔符'	delimiter	指定分隔符
-f n1,n2	fields	分割以后显示第几段内容, 使用 , 分割

```
1 # 截取出3.txt文件中前2行以“:”进行分割的第1,2段内容
2 head -2 3.txt | cut -d ':' -f 1,2 #逗号只表示间隔符，取第1段
  和 第2段
3
4 # 以下方式也可以
5 head -2 3.txt | cut -d ':' -f 1-2 #减号表示区域符，取第1段
  到第2段之间的所有的内容
```

示例3：截取出文件中前2行以“:”进行分割的第2段到最后的内容

范围	含义
n	只显示第n项
n-	显示 从第n项 一直到行尾
n-m	显示 从第n项 到 第m项(包括m)

```
1 # 截取出3.txt文件中前2行以“:”进行分割的第2段到最后的内容
2 head -2 3.txt | cut -d ':' -f 2-
```

wc命令

wc 命令用于显示指定文件 字节数, 单词数, 行数信息。

命令	含义
wc 文件名	显示指定文件字节数, 单词数, 行数信息

示例1：显示指定文件的行数、单词数、字节数信息

```
1 ## 准备工作：创建4.txt文件
2 111
3 222 bbb
4 333 aaa bbb
5 444 aaa bbb ccc
6 555 aaa bbb ccc ddd
7 666 aaa bbb ccc ddd eee
```

```
1 | wc 4.txt
```

- 预期输出效果：

```
1 [root@bigdata-node1 file]# wc 4.txt
2  6 21 85 4.txt
3
4 [root@bigdata-node1 file]# ls -l
5 -rw-r--r-- 1 root root  85 4月 12 23:20 4.txt
```

- 数字6表示：行数
- 数字21表示：单词数
- 数字85表示：字节数

示例2：显示指定文件的行数信息

选项	英文	含义
-c	bytes	字节数
-w	words	单词数

选项	英文	含义
-l	lines	行数

```
1 | wc -l 4.txt
```

示例3: 查看 /etc 目录下有多少个子内容

```
1 | ls /etc | wc -w
```

uniq命令

`uniq` 命令用于检查及删除文本文件中重复出现的行，一般与 `sort` 命令结合使用。

示例1: 实现去重效果

命令	英文	含义
<code>uniq [选项] 文件</code>	unique 唯一	去除重复行

```
1 | ## 准备工作: 创建5.txt文件
2 | 张三      98
3 | 李四      100
4 | 王五      90
5 | 赵六      95
6 | 麻七      70
7 | 李四      100
8 | 王五      90
9 | 赵六      95
10 | 麻七      70
```

```
1 | # 对5.txt文件中按第二列数据进行数值排序(降序), 且去重复
2 | cat 5.txt | sort | uniq
```

示例2: 查看指定目录下的子内容数量

选项	英文	含义
-c	count	统计每行内容出现的次数

```
1 | cat 5.txt | sort -k2nr | uniq -c
```